

PROGRAMA DE CURSO

Introducción a la Programación con Python

∇ Nablabs

NLPY-1

Duración:	14 semanas totales (10 semanas de clases + 4 semanas de proyecto final)
Nivel:	Principiante
Modalidad:	Virtual Sincrónica
Carga horaria:	8 horas semanales (4 horas de clase + 4 horas extraclase recomendadas)

Índice

1. Descripción General del Curso	3
1.1. Metodología de Enseñanza	3
2. Objetivos del Curso	4
2.1. Objetivo General	4
2.2. Objetivos Específicos	4
2.2.1. Fundamentos de Programación	4
2.2.2. Desarrollo de Software	4
2.2.3. Herramientas y Buenas Prácticas	4
2.2.4. Competencias Transversales	5
3. Contenidos por Semana	6
3.1. Semana 0: Conceptos Fundamentales	6
3.2. Semana 1: Condicionales y Control de Flujo	6
3.3. Semana 2: Bucles y Estructuras de Repetición	7
3.4. Semana 3: Excepciones y Manejo de Errores	8
3.5. Semana 4: Librerías y Módulos	8
3.6. Semana 5: Pruebas Unitarias y Testing	9
3.7. Semana 6: Entrada/Salida de Archivos	10
3.8. Semana 7: Expresiones Regulares	11
3.9. Semana 8: Programación Orientada a Objetos (POO)	12
3.10. Semana 9: Conceptos Avanzados	13
4. Evaluación	14
4.1. Distribución de la Calificación	14
4.2. Tareas Semanales (60 %)	14
4.2.1. Entrega de Tareas mediante GitHub	14
4.2.2. Justificación del Modelo de Evaluación	16
4.3. Proyecto Final (30 %)	17
4.4. Participación (10 %)	17
4.5. Escala de Calificación	18
5. Cronograma y Estructura de Sesiones	19
5.1. Distribución Temporal	19
5.2. Estructura de Sesiones Semanales	19
5.2.1. Primera Sesión (2 horas) - Teoría y Práctica	19
5.2.2. Segunda Sesión (2 horas) - Defensas y Complemento Teórico	19
5.3. Cronograma General	20
6. Aspectos Operativos	21
6.1. Modalidad del Curso	21
6.2. Requisitos Técnicos	21
6.2.1. Hardware	21

6.2.2. Software	21
6.2.3. Plataformas	22
6.3. Justificación de Requisitos	22
6.4. Carga de Trabajo	22
6.5. Recursos de Aprendizaje	23
6.5.1. Recursos Obligatorios	23
6.5.2. Recursos Recomendados	23
6.5.3. Herramientas de Apoyo	23
6.6. Políticas del Curso	23
6.6.1. Asistencia	23
6.6.2. Entregas	24
6.6.3. Integridad Académica	24
6.6.4. Comunicación	24
7. Criterios de Aprobación	25
8. Perfil del Egresado	25
8.1. Competencias Técnicas	25
8.2. Competencias Metodológicas	25
8.3. Competencias Comunicativas	26
8.4. Preparación para Continuar Aprendiendo	26
9. Información de Contacto	27
10. Anexos	27
10.1. Anexo A: Calendario de Entregas	27
10.2. Anexo B: Rúbricas de Evaluación	27
10.3. Anexo C: Especificaciones de Tareas Semanales	27
10.4. Anexo D: Especificaciones del Proyecto Final	27
10.5. Anexo E: Tutorial de GitHub para el Curso	27

1 Descripción General del Curso

Este curso proporciona una introducción comprensiva y práctica a la programación utilizando Python como lenguaje principal. Diseñado específicamente para estudiantes sin conocimientos previos de programación, el curso adopta un enfoque pedagógico gradual que combina teoría, ejemplos prácticos y proyectos aplicados.

Python ha sido seleccionado como lenguaje de enseñanza por su sintaxis clara y legible, su amplia aplicabilidad en diversos dominios profesionales (desarrollo web, ciencia de datos, inteligencia artificial, automatización), y por contar con una comunidad global robusta que facilita el aprendizaje continuo.

A lo largo de las 10 semanas de instrucción, los estudiantes desarrollarán competencias fundamentales en pensamiento computacional, resolución de problemas algorítmicos, y buenas prácticas de programación. El curso culmina con un proyecto final integrador que permite a los estudiantes demostrar y consolidar los conocimientos adquiridos mediante el desarrollo de una aplicación funcional.

1.1 Metodología de Enseñanza

El curso implementa una metodología activa y orientada a la práctica:

- **Aprendizaje incremental:** Cada semana construye sobre los conocimientos de la anterior, asegurando una progresión lógica y sostenible.
- **Teoría aplicada:** Los conceptos se presentan siempre acompañados de ejemplos prácticos y casos de uso reales.
- **Aprender haciendo:** Cada semana incluye una tarea práctica que refuerza el aprendizaje mediante la aplicación directa de conceptos.
- **Defensa oral de código:** Los estudiantes deben explicar su código durante las sesiones de defensa, asegurando comprensión profunda y evitando malas prácticas académicas.
- **Control de versiones con Git/GitHub:** Los estudiantes aprenderán buenas prácticas de desarrollo mediante el uso de repositorios Git, documentando su progreso con commits organizados.
- **Retroalimentación continua:** Las defensas semanales proporcionan feedback inmediato y personalizado.

2 Objetivos del Curso

2.1 Objetivo General

Objetivo

Desarrollar competencias fundamentales en programación mediante el aprendizaje de Python, capacitando a los estudiantes para diseñar, implementar y depurar soluciones algorítmicas a problemas computacionales de complejidad básica e intermedia, aplicando principios de pensamiento lógico, buenas prácticas de desarrollo y metodologías de testing.

2.2 Objetivos Específicos

Al finalizar el curso, los estudiantes serán capaces de:

2.2.1 Fundamentos de Programación

- Comprender y aplicar los conceptos fundamentales de programación: variables, tipos de datos, operadores y expresiones.
- Utilizar estructuras de control de flujo (condicionales y bucles) para diseñar algoritmos básicos.
- Manipular diferentes tipos de datos: números, strings, listas, tuplas, diccionarios y conjuntos.

2.2.2 Desarrollo de Software

- Implementar funciones reutilizables aplicando principios de modularización y abstracción.
- Manejar excepciones para crear programas robustos y tolerantes a fallos.
- Trabajar con módulos y librerías de Python, tanto estándar como de terceros.
- Aplicar principios de Programación Orientada a Objetos (POO) para organizar código complejo.

2.2.3 Herramientas y Buenas Prácticas

- Leer y escribir archivos para persistencia de datos.
- Implementar pruebas unitarias para validar el correcto funcionamiento del código.
- Utilizar expresiones regulares para procesamiento avanzado de texto.
- Consumir APIs REST básicas para integración con servicios externos.
- Gestionar entornos virtuales y dependencias de proyectos.

- Utilizar Git y GitHub para control de versiones y documentación de desarrollo.

2.2.4 Competencias Transversales

- Depurar código identificando y corrigiendo errores de sintaxis, lógica y ejecución.
- Documentar código de manera clara y profesional.
- Aplicar técnicas de comprehensions, funciones lambda y generadores para escribir código Python idiomático.
- Desarrollar pensamiento algorítmico para descomponer problemas complejos en soluciones programáticas.

3 Contenidos por Semana

3.1 Semana 0: Conceptos Fundamentales

Objetivo

Familiarizarse con el entorno de Python y dominar los elementos básicos del lenguaje: entrada/salida, variables, tipos de datos primitivos y operaciones aritméticas.

Contenidos:

- ¿Qué es Python? Características y aplicaciones
- Instalación y configuración del entorno (Python, IDE)
- Función `print()`: salida de datos
- Función `input()`: entrada de datos del usuario
- Variables y asignación
- Tipos de datos básicos: strings, integers, floats, booleans
- Comentarios en el código (`#` y `"""`)
- Formateo de strings (`f-strings`, `format()`, concatenación)
- Métodos de strings: `strip()`, `title()`, `capitalize()`, `lower()`, `upper()`, `split()`
- Operadores aritméticos: `+`, `-`, `*`, `/`, `//`, `%`, `**`
- Conversión de tipos (type casting): `int()`, `float()`, `str()`
- Función `type()` para identificar tipos de datos

Proyecto semanal: Calculadora de propinas o programa interactivo simple

3.2 Semana 1: Condicionales y Control de Flujo

Objetivo

Implementar lógica de decisión en programas mediante estructuras condicionales, utilizando operadores de comparación y lógicos para crear flujos de ejecución alternativos.

Contenidos:

- Operadores de comparación: `>`, `<`, `>=`, `<=`, `==`, `!=`
- Valores booleanos: `True` y `False`
- Sentencia `if`: ejecución condicional

- Sentencia **elif**: múltiples condiciones
- Sentencia **else**: caso por defecto
- Operadores lógicos: **and**, **or**, **not**
- Anidamiento de condicionales
- Operador ternario (expresión condicional)
- Operador **match** (pattern matching en Python 3.10+)
- Truthy y Falsy values
- Validación de entrada con condicionales

Proyecto semanal: Sistema de calificaciones o validador de entrada

3.3 Semana 2: Bucles y Estructuras de Repetición

Objetivo

Automatizar tareas repetitivas mediante bucles, comprendiendo cuándo usar **while** vs **for**, y aplicar control de flujo dentro de iteraciones.

Contenidos:

- ¿Qué es un bucle? Conceptos de iteración
- Bucle **while**: iteración condicional
- Bucle **for**: iteración sobre secuencias
- Función **range()**: generación de secuencias numéricas
- Iteración sobre strings, listas y otros iterables
- Sentencia **break**: salir de un bucle
- Sentencia **continue**: saltar a la siguiente iteración
- Sentencia **else** en bucles
- Bucles anidados
- Bucles infinitos y cómo evitarlos
- Validación de entrada con bucles
- Patrones comunes de iteración

Proyecto semanal: Juego de adivinanzas o generador de patrones

3.4 Semana 3: Excepciones y Manejo de Errores

Objetivo

Desarrollar programas robustos mediante el manejo apropiado de excepciones, anticipando y gestionando errores de ejecución de manera controlada.

Contenidos:

- ¿Qué son las excepciones? Diferencia entre errores de sintaxis y de ejecución
- Tipos de errores comunes: `SyntaxError`, `ValueError`, `NameError`, `TypeError`, `ZeroDivisionError`, `IndexError`, `KeyError`
- Bloque `try`: código que puede fallar
- Bloque `except`: captura de excepciones
- Captura de excepciones específicas vs genéricas
- Bloque `else`: código si no hubo excepciones
- Bloque `finally`: código que siempre se ejecuta
- Sentencia `raise`: lanzar excepciones manualmente
- Creación de excepciones personalizadas
- Buenas prácticas en manejo de errores
- Debugging básico

Proyecto semanal: Validador robusto de datos de usuario

3.5 Semana 4: Librerías y Módulos

Objetivo

Ampliar la funcionalidad de los programas mediante el uso de módulos de la biblioteca estándar y librerías de terceros, comprendiendo los mecanismos de importación y organización de código.

Contenidos:

- ¿Qué son módulos y paquetes?
- Diferencia entre módulo, paquete y librería
- Importación de módulos: `import`
- Importación con alias: `import ... as`

- Importación selectiva: `from ... import`
- Módulo `math`: funciones matemáticas avanzadas
- Módulo `random`: generación de números aleatorios
- Módulo `datetime`: manejo de fechas y horas
- Módulo `os`: interacción con el sistema operativo
- Instalación de paquetes con `pip`
- Exploración de PyPI (Python Package Index)
- Creación de módulos propios
- Variable `__name__` y `if __name__ == "__main__"`

Proyecto semanal: Programa que utiliza múltiples módulos

3.6 Semana 5: Pruebas Unitarias y Testing

Objetivo

Garantizar la calidad del código mediante la implementación de pruebas unitarias, aplicando metodologías de testing para validar el comportamiento esperado de funciones y módulos.

Contenidos:

- ¿Qué es el testing? Importancia de las pruebas
- Tipos de pruebas: unitarias, integración, sistema
- Framework `pytest`: instalación y configuración
- Estructura básica de una prueba
- Aserciones con `assert`
- Convenciones de nombrado para tests
- Casos de prueba: normales, límite y excepcionales
- Testing de excepciones con `pytest.raises()`
- Fixtures básicas
- Ejecución de pruebas
- Interpretación de resultados de tests

- Test-Driven Development (TDD) conceptos básicos
- Coverage: medición de cobertura de código

Proyecto semanal: Suite de pruebas para código previo

3.7 Semana 6: Entrada/Salida de Archivos

Objetivo

Implementar persistencia de datos mediante lectura y escritura de archivos, manejando diferentes formatos y aplicando buenas prácticas de gestión de recursos.

Contenidos:

- ¿Por qué trabajar con archivos? Persistencia de datos
- Función `open()`: apertura de archivos
- Modos de apertura: lectura (`r`), escritura (`w`), añadir (`a`), lectura/escritura (`r+`)
- Modos texto vs binario
- Métodos de lectura: `read()`, `readline()`, `readlines()`
- Métodos de escritura: `write()`, `writelines()`
- Cierre de archivos: método `close()`
- Bloque `with`: gestor de contexto para archivos
- Rutas de archivos: absolutas y relativas
- Módulo `os.path`: manipulación de rutas
- Módulo `pathlib`: rutas orientadas a objetos
- Archivos CSV: lectura y escritura
- Módulo `csv`: `DictReader`, `DictWriter`
- Codificación de caracteres (UTF-8)
- Manejo de errores al trabajar con archivos

Proyecto semanal: Agenda de contactos con persistencia

3.8 Semana 7: Expresiones Regulares

Objetivo

Procesar y validar texto de manera avanzada mediante expresiones regulares, aplicando patrones para búsqueda, extracción y manipulación de strings complejos.

Contenidos:

- ¿Qué son las expresiones regulares (regex)?
- Módulo `re`: funciones principales
- Función `search()`: buscar patrón
- Función `match()`: verificar patrón al inicio
- Función `findall()`: encontrar todas las coincidencias
- Función `sub()`: reemplazar con patrón
- Función `split()`: dividir por patrón
- Patrones básicos: literales y metacaracteres
- Clases de caracteres: `\d`, `\w`, `\s`, `\D`, `\W`, `\S`
- Cuantificadores: `*`, `+`, `?`, `{n}`, `{n,m}`
- Anclas: `^`, `$`, `\b`
- Grupos de captura: `()`, grupos nombrados
- Alternación: `|`
- Raw strings: prefijo `r`
- Compilación de patrones: `re.compile()`
- Flags: `re.IGNORECASE`, `re.MULTILINE`
- Casos de uso: validación de emails, teléfonos, URLs

Proyecto semanal: Extractor/validador de información con regex

3.9 Semana 8: Programación Orientada a Objetos (POO)

Objetivo

Diseñar y implementar soluciones utilizando el paradigma de Programación Orientada a Objetos, aplicando conceptos de encapsulación, herencia y polimorfismo para organizar código complejo de manera modular y reutilizable.

Contenidos:

- ¿Qué es la POO? Paradigmas de programación
- Conceptos fundamentales: clases y objetos
- Definición de clases con `class`
- Método `__init__()`: constructor
- Parámetro `self`: referencia a la instancia
- Atributos de instancia vs atributos de clase
- Métodos de instancia
- Encapsulación: convenciones de privacidad (`_`, `__`)
- Propiedades con decorador `@property`
- Setters con `@propiedad.setter`
- Métodos especiales (dunder methods): `__str__`, `__repr__`, `__eq__`, `__add__`, etc.
- Herencia: clases padre e hija
- Función `super()`: acceso a la clase padre
- Sobrescritura de métodos (method overriding)
- Métodos de clase: decorador `@classmethod`
- Métodos estáticos: decorador `@staticmethod`
- Composición vs herencia
- Principios SOLID (introducción)

Proyecto semanal: Sistema de gestión (biblioteca, inventario, estudiantes)

3.10 Semana 9: Conceptos Avanzados

Objetivo

Aplicar técnicas avanzadas de Python para escribir código más eficiente, expresivo y pythonic, incluyendo comprehensions, generadores, decoradores y consumo de APIs.

Contenidos:

- Conjuntos (`set`): operaciones y usos
- Métodos avanzados de diccionarios
- Funciones lambda: sintaxis y aplicaciones
- Funciones `map()`, `filter()`, `reduce()`
- List comprehensions: sintaxis y casos de uso
- Dictionary comprehensions
- Set comprehensions
- Argumentos variables: `*args` y `**kwargs`
- Desempaquetado de argumentos
- Generadores: palabra clave `yield`
- Expresiones generadoras vs listas
- Iteradores: protocolo de iteración
- Decoradores: concepto y sintaxis básica
- Decoradores con argumentos
- Uso práctico de decoradores (timing, logging)
- Introducción a APIs REST
- Librería `requests`: GET, POST, PUT, DELETE
- Procesamiento de respuestas JSON
- Manejo de códigos de estado HTTP
- Entornos virtuales: `venv`, `virtualenv`
- Gestión de dependencias: `requirements.txt`
- Buenas prácticas de desarrollo

Proyecto semanal: Analizador de datos desde API

4 Evaluación

4.1 Distribución de la Calificación

La calificación final del curso se distribuye de la siguiente manera:

Componente	Porcentaje	Descripción
Tareas Semanales	60 %	10 tareas (Semanas 0-9), 6 % cada una
Proyecto Final	30 %	Proyecto integrador individual
Participación	10 %	Asistencia y quizzes

Cuadro 1: Distribución de calificación

4.2 Tareas Semanales (60 %)

Cada semana (de la 0 a la 9) incluye una tarea práctica que debe ser entregada y defendida. La evaluación de cada tarea se divide en:

- **Código (40 %):** Evaluación del código fuente entregado considerando:
 - Funcionalidad: el programa cumple con los requisitos especificados
 - Corrección: el código ejecuta sin errores
 - Calidad: aplicación de buenas prácticas, código limpio y legible
 - Organización: estructura lógica y modular del código
 - Documentación: comentarios y docstrings apropiados
- **Defensa (60 %):** Presentación oral donde el estudiante debe:
 - Explicar la lógica y funcionamiento de su código
 - Responder preguntas sobre decisiones de diseño e implementación
 - Demostrar comprensión de los conceptos aplicados
 - Identificar y explicar secciones específicas del código
 - Proponer mejoras o alternativas de implementación

4.2.1 Entrega de Tareas mediante GitHub

Información

Todas las tareas semanales deben entregarse a través de GitHub siguiendo el protocolo establecido.

Repositorio de Tareas Semanales:

1. Cada estudiante creará un repositorio público en GitHub con el nombre: `NLPY-1-nombreEstudiante`

2. El repositorio debe incluir en su descripción: “Repositorio de entregas del curso NLPY-1 (Introducción a la Programación con Python) de Nablab”
3. Estructura del repositorio:

```
NLPY-1-nombreEstudiante/  
  README.md  
  S0/  
    tarea0.py  
    README.md (opcional)  
  S1/  
    tarea1.py  
    README.md (opcional)  
  S2/  
    ...  
  S9/  
    ...
```

4. Cada carpeta SX (donde X es el número de semana) debe contener todo el código y archivos relacionados con la tarea de esa semana.
5. El README.md principal debe incluir:
 - Nombre del estudiante
 - Información del curso (NLPY-1)
 - Índice de tareas con enlaces a cada carpeta

Repositorio del Proyecto Final:

1. Crear un repositorio separado para el proyecto final: NLPY-1-ProyectoFinal-nombreEstudiante
2. El repositorio debe incluir en su descripción: “Proyecto Final del curso NLPY-1 (Introducción a la Programación con Python) de Nablab”
3. Debe contener:
 - Todo el código fuente bien estructurado
 - README.md completo con descripción del proyecto, instrucciones de instalación y uso
 - requirements.txt con todas las dependencias
 - Documentación apropiada
 - Carpeta de tests (si aplica)

 Nota Importante**Importancia de los Commits Organizados**

Los commits en GitHub son una evidencia fundamental del proceso de desarrollo y aprendizaje. Se espera que los estudiantes:

- Realicen **commits frecuentes y organizados** a medida que avanzan en el desarrollo de sus tareas y proyecto.
- Cada commit debe tener un **mensaje descriptivo** que explique claramente qué cambios se realizaron.
- El historial de commits debe mostrar **progreso incremental**, reflejando el proceso de desarrollo paso a paso.
- **Un solo commit masivo con todo el código** es indicador de actividad fraudulenta (plagio, copia, o trabajo realizado por terceros) y será motivo de investigación académica.
- Los commits deben ser **regulares y distribuidos en el tiempo**, no concentrados en las últimas horas antes de la entrega.

Ejemplos de buenos commits:

- “Implementa función de validación de entrada”
- “Agrega manejo de excepciones para división por cero”
- “Refactoriza código de bucle principal”
- “Corrige error en cálculo de promedio”
- “Agrega documentación a funciones principales”

El historial de commits forma parte de la evaluación del código y puede ser revisado durante la defensa oral. Los estudiantes deben estar preparados para explicar el proceso de desarrollo reflejado en sus commits.

4.2.2 Justificación del Modelo de Evaluación

La ponderación mayor en la defensa oral (60%) responde a objetivos pedagógicos fundamentales:

1. **Garantizar aprendizaje genuino:** El estudiante debe comprender profundamente su código, no solo ejecutarlo.
2. **Prevenir plagio:** La explicación oral evidencia autoría y comprensión real del trabajo entregado.

3. **Evitar uso inadecuado de IA:** Si bien las herramientas de IA pueden asistir en el aprendizaje, la defensa asegura que el estudiante comprende y puede explicar cada línea de código.
4. **Desarrollar competencias comunicativas:** Explicar código técnicamente es una habilidad profesional esencial.
5. **Retroalimentación inmediata:** La defensa permite identificar y corregir malentendidos conceptuales en tiempo real.

Modalidad de defensa:

- Sesión individual de 5-10 minutos por estudiante
- Realizada en la segunda sesión de cada semana
- El estudiante presenta su código y responde preguntas del instructor
- Se evalúa en tiempo real durante la sesión

4.3 Proyecto Final (30 %)

El proyecto final es un trabajo integrador individual que se desarrolla durante las semanas 10-13, con entrega y presentación en la semana 14. Requiere aplicar múltiples conceptos del curso en una aplicación funcional completa.

Criterios de evaluación:

- Complejidad y completitud de la solución (30 %)
- Aplicación correcta de conceptos de POO (20 %)
- Manejo de errores y robustez (15 %)
- Calidad del código y buenas prácticas (15 %)
- Documentación y README (10 %)
- Presentación y defensa del proyecto (10 %)

(Las especificaciones detalladas del proyecto se proporcionan en documento aparte)

4.4 Participación (10 %)

Evaluación continua basada en:

- **Asistencia (5 %):** Presencia y puntualidad en sesiones sincrónicas
 - 90-100 % de asistencia: 5 %
 - 80-89 % de asistencia: 4 %

- 70-79 % de asistencia: 3 %
 - Menos de 70 %: 0 %
- **Quizzes y actividades en clase (5 %):** Ejercicios cortos durante las sesiones para verificar comprensión de conceptos

4.5 Escala de Calificación

Se utiliza la escala estándar del 0 al 100:

Rango	Calificación
90-100	Excelente
80-89	Muy bueno
70-79	Bueno
60-69	Satisfactorio
0-59	Insuficiente

Nota mínima aprobatoria: 70

5 Cronograma y Estructura de Sesiones

5.1 Distribución Temporal

Fase de Instrucción: Semanas 1-10 (10 semanas)

- 2 sesiones por semana
- 4 horas totales de clase semanal
- Tarea semanal asignada

Fase de Proyecto: Semanas 11-14 (4 semanas)

- Desarrollo individual del proyecto final
- Sesiones de consulta y seguimiento
- Entrega y presentación final en semana 14

5.2 Estructura de Sesiones Semanales

5.2.1 Primera Sesión (2 horas) - Teoría y Práctica

- **Duración:** 2 horas completas
- **Formato:** Teoría y ejemplos
- **Contenido:**
 - Presentación de conceptos nuevos (45-60 min)
 - Ejemplos demostrativos en vivo (30-40 min)
 - Ejercicios guiados grupales (20-30 min)
 - Asignación de tarea semanal (10 min)

5.2.2 Segunda Sesión (2 horas) - Defensas y Complemento Teórico

- **Duración:** 2 horas totales
- **Formato:** Mixto
- **Contenido:**
 - Complemento teórico y aclaraciones (40 min)
 - Defensas de tareas individuales (80 min)
 - 5-10 minutos por estudiante
 - El estudiante presenta su código
 - Responde preguntas del instructor
 - Recibe retroalimentación inmediata

5.3 Cronograma General

Semana	Tema	Entregable
0	Conceptos Fundamentales	Tarea 0
1	Condicionales y Control de Flujo	Tarea 1
2	Bucles y Estructuras de Repetición	Tarea 2
3	Excepciones y Manejo de Errores	Tarea 3
4	Librerías y Módulos	Tarea 4
5	Pruebas Unitarias y Testing	Tarea 5
6	Entrada/Salida de Archivos	Tarea 6
7	Expresiones Regulares	Tarea 7
8	Programación Orientada a Objetos	Tarea 8
9	Conceptos Avanzados	Tarea 9
10-13	Desarrollo de Proyecto Final	Avances semanales
14	Presentaciones Finales	Entrega Proyecto Final

Cuadro 2: Cronograma del curso

6 Aspectos Operativos

6.1 Modalidad del Curso

Formato: Virtual Sincrónico

Las clases se imparten en tiempo real a través de plataforma de videoconferencia, permitiendo interacción directa entre instructor y estudiantes. Se requiere participación activa durante las sesiones programadas.

Ventajas de la modalidad sincrónica:

- Interacción en tiempo real con el instructor
- Resolución inmediata de dudas
- Aprendizaje colaborativo con compañeros
- Defensas de tareas en vivo para feedback personalizado
- Sesiones de programación en vivo (live coding)

6.2 Requisitos Técnicos

Para participar efectivamente en el curso, cada estudiante debe contar con:

6.2.1 Hardware

- **Computadora:** PC, laptop, Mac, o computadora con Linux
- **Procesador:** Mínimo dual-core (recomendado i5 o superior)
- **RAM:** Mínimo 4 GB (recomendado 8 GB o más)
- **Almacenamiento:** Al menos 10 GB de espacio libre en disco
- **Conectividad:** Acceso estable a Internet (recomendado mínimo 5 Mbps)
- **Periféricos:** Micrófono y cámara web (para participación en sesiones)

6.2.2 Software

Los estudiantes deberán instalar (con guía proporcionada):

- **Python 3.10 o superior**
- **Editor de código o IDE:** Visual Studio Code, PyCharm, Sublime Text, o similar
- **Git:** Para control de versiones
- **Cuenta en GitHub:** Para entrega de tareas y proyecto
- **Navegador web moderno:** Chrome, Firefox, Edge, o Safari

6.2.3 Plataformas

- Acceso a plataforma de videoconferencia del curso (Zoom, Google Meet, Microsoft Teams, según se indique)
- Cuenta en GitHub (gratuita)
- Cuenta en plataforma de gestión del curso (si aplica)

6.3 Justificación de Requisitos

¿Por qué se requiere una computadora con sistema operativo completo?

1. **Entorno de desarrollo real:** Los estudiantes trabajarán en el mismo tipo de ambiente que utilizarán profesionalmente.
2. **Instalación de herramientas:** Python y sus librerías requieren instalación y configuración en el sistema operativo.
3. **Gestión de archivos:** Las tareas incluyen lectura/escritura de archivos, operaciones con sistema de archivos, y gestión de proyectos.
4. **Terminal/consola:** Muchas operaciones requieren uso de línea de comandos.
5. **Debugging y testing:** Herramientas de depuración y testing funcionan mejor en entornos de desarrollo completos.
6. **Control de versiones:** Git requiere instalación local y gestión de repositorios.
7. **Proyecto final:** El proyecto requiere capacidad de crear aplicaciones con múltiples archivos, módulos y dependencias.

Nota Importante

No es posible completar satisfactoriamente el curso utilizando únicamente dispositivos móviles (tablets, smartphones) o entornos de navegador web, ya que estos limitan significativamente las capacidades de desarrollo, instalación de herramientas, uso de Git, y ejecución de proyectos complejos.

6.4 Carga de Trabajo

Tiempo en clase: 4 horas semanales

- Primera sesión: 2 horas (lunes/martes)
- Segunda sesión: 2 horas (jueves/viernes)

Tiempo extraclasses recomendado: 4 horas semanales

- Realización de tarea semanal: 2-3 horas

- Estudio y práctica adicional: 1-2 horas
- Preparación para defensa de tarea: 30-60 minutos
- **Total estimado:** 8 horas semanales de dedicación
- **Durante fase de proyecto (semanas 10-14):**
- Se recomienda dedicar 6-8 horas semanales al desarrollo del proyecto
- Sesiones de consulta disponibles según cronograma

6.5 Recursos de Aprendizaje

6.5.1 Recursos Obligatorios

- Presentaciones de clase (proporcionadas semanalmente)
- Especificaciones de tareas (documento por semana)
- Material de lectura complementario (según se indique)

6.5.2 Recursos Recomendados

- Documentación oficial de Python: <https://docs.python.org/es/>
- Tutorial oficial de Python: <https://docs.python.org/es/3/tutorial/>
- Real Python: <https://realpython.com/>
- Python Tutor (visualizador de código): <https://pythontutor.com/>
- Stack Overflow: Para consultas y resolución de problemas
- GitHub Docs: <https://docs.github.com/es>

6.5.3 Herramientas de Apoyo

- Plataformas de práctica: LeetCode, HackerRank, Codewars
- Entornos online (para práctica rápida): Replit, Google Colab
- Foros y comunidades: Reddit r/learnpython, Discord de Python

6.6 Políticas del Curso

6.6.1 Asistencia

- Se requiere asistencia a las sesiones sincrónicas
- La asistencia impacta directamente la calificación de participación
- Faltas justificadas deben notificarse con anticipación

6.6.2 Entregas

- Las tareas deben entregarse antes de la fecha y hora límite especificada
- La entrega consiste en compartir el enlace del repositorio de GitHub
- Entregas tardías tendrán penalización según políticas a definir
- No se aceptan entregas después de realizada la defensa
- El repositorio debe estar público para permitir revisión

6.6.3 Integridad Académica

- Se espera trabajo individual en todas las tareas y proyecto
- El plagio o copia resultará en calificación de cero
- El uso de IA generativa está permitido como herramienta de aprendizaje, pero el estudiante debe poder explicar y defender todo su código
- La defensa oral es obligatoria y determinante para la calificación
- El historial de commits en GitHub será revisado como evidencia de trabajo propio
- Commits sospechosos (un solo commit masivo, commits copiados, etc.) serán investigados

6.6.4 Comunicación

- Canal oficial de comunicación: [especificar plataforma]
- Tiempo de respuesta a consultas: 24-48 horas hábiles
- Horarios de consulta: [especificar según disponibilidad]

7 Criterios de Aprobación

Para aprobar el curso, el estudiante debe:

1. Obtener una calificación final mínima de **70 puntos** (sobre 100)
2. Completar y defender satisfactoriamente al menos **8 de 10 tareas semanales**
3. Entregar y presentar el **proyecto final**
4. Mantener al menos **70 % de asistencia** a las sesiones sincrónicas
5. Contar con repositorios de GitHub activos con historial de commits apropiado

8 Perfil del Egresado

Al completar exitosamente este curso, el estudiante habrá desarrollado las siguientes competencias:

8.1 Competencias Técnicas

- Escribir programas funcionales en Python aplicando sintaxis correcta
- Diseñar algoritmos para resolver problemas computacionales
- Implementar estructuras de datos apropiadas para diferentes contextos
- Desarrollar aplicaciones orientadas a objetos con diseño modular
- Crear y ejecutar pruebas unitarias para validar código
- Manejar archivos y persistencia de datos
- Consumir APIs REST básicas
- Utilizar Git y GitHub para control de versiones
- Gestionar entornos virtuales y dependencias

8.2 Competencias Metodológicas

- Descomponer problemas complejos en componentes manejables
- Depurar código sistemáticamente
- Aplicar pensamiento algorítmico y lógico
- Leer y comprender documentación técnica
- Investigar y aprender nuevas librerías de forma autónoma
- Documentar procesos de desarrollo mediante commits organizados

8.3 Competencias Comunicativas

- Explicar código y decisiones de diseño técnicamente
- Documentar código de manera clara y profesional
- Presentar proyectos técnicos efectivamente
- Escribir mensajes de commit descriptivos y profesionales

8.4 Preparación para Continuar Aprendiendo

El egresado estará preparado para:

- Explorar frameworks web (Django, Flask)
- Iniciarse en ciencia de datos (Pandas, NumPy, Matplotlib)
- Aprender desarrollo de APIs (FastAPI, REST)
- Continuar con programación avanzada y patrones de diseño
- Participar en proyectos open source
- Colaborar en proyectos mediante GitHub
- Desarrollar proyectos personales o profesionales con Python

9 Información de Contacto

▽ Nablab

Curso: Introducción a la Programación con Python (NLPY-1)

Nivel: Principiante

Para más información, consulte los canales oficiales del curso

10 Anexos

10.1 Anexo A: Calendario de Entregas

(A completar con fechas específicas al inicio del curso)

10.2 Anexo B: Rúbricas de Evaluación

(Documentos detallados de criterios de evaluación por tarea)

10.3 Anexo C: Especificaciones de Tareas Semanales

(Documentos individuales por semana)

10.4 Anexo D: Especificaciones del Proyecto Final

(Documento detallado con requisitos del proyecto integrador)

10.5 Anexo E: Tutorial de GitHub para el Curso

(Guía paso a paso para crear repositorios y realizar commits)

Última actualización: Octubre 2025

Versión: 1.0

Notas Finales

Este programa de curso está diseñado para proporcionar una experiencia de aprendizaje estructurada, práctica y orientada al desarrollo de competencias reales en programación. La combinación de teoría, práctica supervisada, defensa oral de código, y uso de control de versiones garantiza que los estudiantes no solo aprendan a programar, sino que comprendan profundamente los conceptos y puedan aplicarlos de manera autónoma.

El enfoque en la defensa oral (60 % de cada tarea) refleja nuestro compromiso con el aprendizaje genuino y la formación de programadores que comprenden su código, no solo lo ejecutan. Esta metodología prepara a los estudiantes para entornos profesionales donde la capacidad de explicar y defender decisiones técnicas es tan importante como la habilidad de escribir código.

La integración de GitHub como plataforma de entrega introduce a los estudiantes en herramientas profesionales de desarrollo, fomentando buenas prácticas desde el inicio de su formación. El historial de commits no solo sirve como evidencia de trabajo propio, sino como registro del proceso de aprendizaje y evolución como programadores.

¡Bienvenidos al curso y éxito en su viaje de aprendizaje en programación!